EXERCICE RÉCAPITULATIF — GIT & REACT

Tu développes une application React.Le dépôt distant est déjà créé sur GitHub. Tu dois récupérer le projet, développer une fonctionnalité, sauvegarder tes changements, travailler avec des branches et corriger quelques erreurs sans perdre inutilement ton travail.

Consigne : pour chaque question, écris la commande Git adaptée. Plusieurs commandes peuvent être nécessaires.

A. Récupérer et synchroniser le projet

1. Tu n'as pas encore le projet sur ton ordinateur. Quelle commande permet de copier le dépôt distant en local ?
2. Tu es dans le projet local. Quelle commande permet d'afficher les dépôts distants configurés ?
3. Ton collègue a envoyé de nouveaux commits sur la branche distante. Quelle commande récupère et intègre ces changements dans ta branche courante ?
4. Tu as terminé une fonctionnalité et créé un commit local. Quelle commande envoie tes commits vers le dépôt distant ?

B. Sauvegarder les modifications

1. Tu as modifié `src/App.jsx`. Quelle commande place ce fichier dans le staging ?
2. Tu veux placer toutes les modifications suivies du projet dans le staging. Quelle commande utilises-tu ?
3. Les changements sont dans le staging. Quelle commande crée un commit avec le message `Ajout du filtre des tâches` ?
4. Tu as créé un commit local, mais tu ne l'as pas encore envoyé sur GitHub. Quelle commande publie ce commit ?

C. Annuler ou corriger une erreur

1. Tu as modifié `src/App.jsx`, mais tu n'as pas fait `git add`. Tu veux abandonner ces modifications locales. Quelle commande utilises-tu ?
2. Tu as fait `git add src/App.jsx`, mais tu veux retirer ce fichier du staging tout en gardant ses modifications sur le disque. Quelle commande utilises-tu ?
3. Tu as ajouté `debug.log` au suivi Git et tu veux le supprimer du disque tout en préparant sa suppression pour le prochain commit. Quelle commande utilises-tu ?
4. Tu veux supprimer manuellement un fichier du disque avec la commande du système, sans préparer automatiquement sa suppression dans Git. Quelle commande utilises-tu ?
5. Un commit déjà partagé contient une erreur. Tu veux annuler ses changements en conservant l'historique et en créant un nouveau commit. Quelle commande utilises-tu ?
6. Tu es sûr de vouloir abandonner toutes les modifications non commitées des fichiers suivis et remettre le staging et les fichiers suivis à l'état du dernier commit. Quelle commande destructive correspond à ce besoin ?

D. Gérer les branches

1. Tu veux afficher les branches locales existantes. Quelle commande utilises-tu ?
2. Tu veux créer une branche nommée `feature/search` sans changer de branche. Quelle commande utilises-tu ?
3. Tu veux passer sur la branche `feature/search`. Quelle commande moderne utilises-tu ?
4. Dans un ancien tutoriel, on te demande de créer et de passer immédiatement sur `feature/header` avec une seule commande `git checkout`. Quelle commande écris-tu ?

5. Tu veux créer puis rejoindre immédiatement `feature/profile` avec `git switch`. Quelle commande utilises-tu ?
6. Tu es sur `main` et tu veux revenir à la branche `feature/search`. Donne une commande avec `git checkout` et une autre avec `git switch`.

E. Mini-situation finale

Tu as terminé `feature/search`. Tu veux intégrer le dernier travail distant, vérifier le dépôt distant, préparer les fichiers, créer un commit et envoyer la branche sur GitHub.

1. Quelle commande récupère et intègre les changements de la branche distante courante ?
2. Quelles commandes utilises-tu pour ajouter les changements, créer le commit `Ajout de la recherche` puis pousser la branche ?